ENGINEERING HISTORY

VirtualGlove Engineering Journey

One week of hypotheses, measurements, experiments, and play tests.

2026 EDITION / IAIN BENNETT / MIT LICENSE

This document records how VirtualGlove was developed and validated from the first camera prototype through the current playable system. It is deliberately chronological: each milestone describes the problem at that point, the decision made, the evidence gathered, and what changed next.

For the system as it exists today, read Architecture and flows. For exact production performance evidence, read Native movement response and validation. To reproduce supported measurements, use the version-matched Engineering Toolkit.

Skip to a milestone

The week at a glance

Date	Milestone	Decision that moved the project forward
2-3 September	Camera recognition and joystick control	Build a dependable FCEUmm path before attempting native emulation.
4-5 September	Shared recognition and native input	Use one recognition layer; validate the custom Nestopia protocol against the ROM rather than treating notes as gospel.
6 September	End-to-end measurement	Measure capture, inference, transport, receiver, core, and display as separate stages.
6-7 September	Movement experiments	Compare smoothing approaches with repeatable traces and keep every experiment reversible.
7-8 September	MediaPipe-first X/Y	Prefer the newest measured coordinate over optical flow, prediction, or a settling tail.
8-9 September	Fast-sweep continuity	Treat backward jumps as detector/reacquisition problems, not merely smoothing problems.
9 September	Runtime and GPU research	Promote the faster complete CPU graph; reject accelerators that lost end-to-end.
9-10 September	Camera delivery and recovery	Make capture tunable and recovery prove that a real frame can be read.
10 September	Production boundary	Stop tuning thresholds that could not reduce the measured palm-detector cost.
10-11 September	Productization	Simplify controls, strengthen installation and recovery, and release as VirtualGlove.

Milestone 1 - Establish camera recognition and joystick output - 2-3 September 2026

The first objective was not native Power Glove emulation. It was to prove that a camera could produce useful, repeatable NES controls. MediaPipe Hands supplied the hand landmarks; VirtualGlove interpreted position, finger curl, wrist roll, push, and pull. A virtual controller on RetroPie sent ordinary NES-compatible directions and buttons through FCEUmm.

This reliable fallback shaped the rest of the project. Recognition and game output were separated early: one global interpretation of the player's hand could feed Programs A-I and dedicated game profiles without recalibrating for every ROM. Three-dimensional hand geometry replaced weaker two-dimensional finger tests, and concentrated bends were distinguished from ordinary hand motion.

The key decision was to keep FCEUmm working while deeper native work remained uncertain. That made every later experiment recoverable and allowed Super Glove Ball to remain playable in joystick mode.

Milestone 2 - Add shared recognition and native Power Glove input - 4-5 September 2026

Super Glove Ball exposed the limitation of joystick emulation: its native mode expects continuous hand coordinates and Power Glove state, not just mapped D-pad presses. A separately named `lr-nestopia-powerglove` core was therefore built while stock Nestopia and FCEUmm remained untouched.

Protocol work followed an explicit evidence order:

1. The exact supplied Super Glove Ball ROM and its input routines.
2. Controlled emulator traces of writes, reads, assembled bytes, and timing.
3. Visible game behaviour, including controller detection and coordinate motion.
4. Nestopia's existing Power Glove implementation.
5. Manuals and NESdev reverse-engineering notes as supporting hypotheses.

That order prevented a convenient packet description from becoming an unquestioned specification. The custom core eventually carried native X/Y and the recognized hand actions used by the game, while an explicit FCEUmm launch continued to select joystick behaviour.

The same period expanded Glove Academy and calibration. Neutral centre, scale, wrist orientation, reach, jitter, fingers, rolls, push, pull, and safety poses became shared recognition data. Game profiles remained output mappings rather than separate recognition models. This was the architectural boundary that allowed Academy practice, FCEUmm play, and native Nestopia play to agree about what the hand was doing.

Milestone 3 - Measure the complete path - 6 September 2026

Early play tests described movement as responsive in some moments and delayed in others. Treating that as one undifferentiated "lag" problem would have led to guesswork. The pipeline was split into measurable stages:

1. Camera exposure and delivery.
2. Frame dequeue, decode, and preparation.
3. MediaPipe inference and detector reacquisition.
4. Recognition and native-coordinate selection.
5. Signed UDP transmission.
6. RetroPie receipt and native-state publication.
7. Custom-core consumption on the next emulated frame.
8. Emulator and display presentation.

Finite diagnostic traces, saved clips, headless replay, a dot-test utility, and external high-frame-rate video were developed for different questions. Saved clips made software comparisons repeatable. Live game tests supplied the human judgement that a trace cannot: whether the robo-glove felt attached to the hand.

Transport and receiver work quickly fell to roughly millisecond-scale stages. Camera delivery, MediaPipe inference, and the more expensive palm-detector path became the meaningful optimization targets.

Milestone 4 - Explore smoothing and movement response - 6-7 September 2026

The original error-driven smoothing

The first native movement engine adjusted its follow rate from the distance between filtered output and the newest measurement. An ideal 60 Hz step model produced the following historical results:

Normalized X step	Time from first step sample to 95% of target
0.010	200 ms
0.025	200 ms
0.050	167 ms
0.100	Immediate
0.200	Immediate

This explained why large moves could feel prompt while fine aiming retained a settling tail. The model excluded capture, inference, network, emulator, and display time, so its values could not be added to other independent percentiles.

Boost and extrapolation trials

A medium-jump experiment increased the error boost and temporarily allowed a bounded `1.30` extrapolation cap. It reduced modeled settling for some steps but could jump beyond the measured hand. Forward overshoot was visible in play and was rejected. Historical configuration fields remain readable for compatibility, but production movement never extrapolates beyond measured position.

Six live configurations were then traced:

Minimum / boost	Vision events	Valid %	Losses	Age p95	Medium settling p95
0.70 / 4	834	69.18	14	104.5 ms	0.0 ms
0.70 / 8	751	56.72	17	175.0 ms	0.0 ms
0.70 / 12	895	90.17	7	87.2 ms	84.6 ms
0.85 / 4	884	89.37	4	104.1 ms	114.0 ms
0.85 / 8	886	97.74	1	85.4 ms	96.9 ms
0.85 / 12	897	97.44	2	83.1 ms	63.8 ms

The physical movements differed too much to select a winner. A controlled three-window follow-up narrowed the question:

Configuration	Valid %	Losses	Age p95	X error p50 / p95	Medium settling p95
0.70 / 8	94.47	6	87.5 ms	0.0011 / 0.0122	120.8 ms
0.85 / 8	92.87	5	91.5 ms	0.0006 / 0.0052	17.9 ms

Configuration	Valid %	Losses	Age p95	X error p50 / p95	Medium settling p95
0.85 / 12	90.27	7	111.5 ms	0.0004 / 0.0041	112.1 ms

`0.85 / 8` was a useful intermediate candidate, but play testing found it too floaty. This was an important result: smaller numeric error did not necessarily produce the best controller.

Bounded speed-sensitive response

The next design measured velocity from consecutive raw coordinates and capture timestamps. Calibrated reach normalized speed, and measured X/Y jitter created an elliptical resting region. One two-dimensional follow weight rose from `0.70` to `1.00`; stops and meaningful reversals settled promptly, and output could never cross beyond the newest measurement.

The curve corrected two mistakes uncovered during deployment: its original timing assumed 60 coordinate updates per second when the Controller was then producing roughly 9-10 inference results, and saturated saved jitter could be misread as an enormous dead zone. Temporal normalization and a safe fallback fixed those faults. Even corrected, the bounded mode did not feel as directly attached as Latest coordinate and was ultimately retired from the user interface.

Milestone 5 - Make MediaPipe and Latest coordinate authoritative - 7-8 September 2026

Optical flow was tested as a way to fill time between MediaPipe results. In practice its green tracking point could diverge from the visible hand, movement was jerky, and frequent fallback meant it did not provide dependable additional coordinates. It was archived rather than promoted.

The production design became simpler: MediaPipe is authoritative for both landmarks and X/Y, and **Latest coordinate** publishes every newest valid, reach-clamped palm anchor directly. It adds no temporal average, interpolation, prediction, dead zone, replay queue, or emulator-side smoothing.

VirtualGlove retained a calibration-compatible five-point wrist/knuckle anchor. MediaPipe had already calculated all 21 landmarks, so averaging fewer points would not reduce neural-network work. Alternative palm anchors were benchmarked for pose-induced motion, but none earned a production change without sacrificing travel or compatibility.

The runtime also began using the frame's real capture timestamp instead of inference-start time, rejected malformed landmark geometry, clamped coordinates before response processing, cleared history on loss or stale input, and resumed from the first fresh clamped coordinate. Per-player reach tuning changed how much physical travel filled the game area without changing the camera aspect ratio or gesture thresholds.

Historical samples showed why controlled trials remained necessary:

30-second window	Valid Controller samples	Observed tracking losses
Earlier flow revision, dot movement	579/591 (98.0%)	6
Recognition fallback, fast dot movement	447/589 (75.9%)	34
Recognition fallback, actual gameplay	568/590 (96.3%)	3

These were different movements and could not serve as a fair A/B comparison. They did reveal that receiver publication count was not the same as the number of new recognized positions or displayed frames.

Milestone 6 - Improve fast sweeps, recovery, and transport - 8-9 September 2026

Long diagonal sweeps exposed a distinctive failure: the on-screen glove moved forward, jumped backward along a similar line, and then caught up. The problem was not ordinary smoothing. MediaPipe had lost its tracked hand region and returned through the palm detector with a contradictory intermediate result.

A direction-aware search region was tested, then retained after reboot-separated play showed that it improved continuity. A small reacquisition guard holds one result that strongly contradicts established motion or is unusually distant without forward alignment. Strongly aligned forward recovery passes immediately. This avoids the old compound catch-up jump without predicting beyond the hand.

Fast-sweep work also ensured that the newest available camera frame wins, that Dashboard statistics and housekeeping stay outside the inference path, and that native state is published before unrelated virtual-gamepad work on RetroPie. Signed latest-only UDP sessions added authentication and restart safety without creating a movement queue. Emulator-aware selection made native behaviour apply only to Super Glove Ball with `lr-nestopia-powerglove`; every other emulator and game combination uses joystick output.

The trace format grew alongside the questions. It records capture sequence and timestamp, recognition completion, accepted anchor, selected and mapped X/Y, fallback or invalid reason, and transport sequence. Repeated sources are identifiable and cannot be counted as fresh velocity measurements. Tracing is finite, buffered, asynchronous, opt-in, and never records video.

Milestone 7 - Evaluate CPU and GPU inference - 9 September 2026

Understanding the graph

The MediaPipe graph is the complete hand-processing pipeline, not a single neural network. It connects frame preparation, palm detection, hand-region geometry, the landmark model, tracking between detections, and result delivery. During continuity, the landmark model reuses the preceding region. When that evidence fails, the graph invokes the more expensive palm detector.

Promoting MediaPipe 0.10.35

The ARM64/Python 3.12 MediaPipe 0.10.35 runtime with four XNNPACK CPU threads, fused full-colour preparation, 0.35 tracking confidence, a 2.25 search region, and Latest-coordinate output beat the earlier complete runtime:

Runtime	Inference p50 / p95	Continuity	One-frame misses
Historical 0.10.18	36.12 / 60.18 ms	97.96%	15
Shipped 0.10.35	34.03 / 46.60 ms	97.96%	15

Landmark-continuation p95 fell from 51.10 to 44.29 ms, while palm-reacquisition p95 fell from 149.48 to 120.16 ms. A live Super Glove Ball trace improved capture-to-send p95 from 126.84 to 91.16 ms with no lost transport samples. A ten-minute complete-camera soak reached 61.7 °C, remained thermally stable, and held memory near a 484 MiB plateau after initialization.

Giving the Adreno GPU a fair test

The UNO Q's Adreno 702 at 845 MHz was reached through Mesa Turnip using EGL/OpenGL ES, OpenCL through Rusticl, and Vulkan. The GPU was real and usable; the complete hand workloads simply did not suit the tested delegate paths.

Lane	CPU result	Adreno result	Decision
MediaPipe hand landmark continuation	32.74 ms p50	384.12 ms p50	GPU rejected
MNN landmark-lite	19.7 / 26.8 ms	43.9 / 44.0 ms	Vulkan slower and numerically incompatible
MNN palm-lite	36.3 / 44.3 ms	77.9 / 78.2 ms	Vulkan slower and numerically incompatible
ncnn exact landmark model	25.0 / 38.7 ms	385.5 / 386.9 ms	Vulkan slower and numerically incompatible

The MediaPipe OpenGL graph repeatedly synchronized small tensors. MNN profiling found work spread across convolution and depthwise-convolution layers rather than one removable tail. An ncnn CPU sidecar reproduced the landmark model but lost its model-only advantage to image preparation, handoff, and duplicated tracking geometry: complete replay was 36.1-36.5 ms p50, 45.7-52.3 ms p95, and 98.68% continuous.

The decision was therefore CPU before exotic acceleration. A future Qualcomm QNN lane must be redistributable and prove at least 20% lower end-to-end p95, ordered newest-sample delivery, recognition within one percentage point, no extra jitter or false gestures, and ten-minute thermal stability.

Milestone 8 - Refine camera delivery and resilience - 9-10 September 2026

With inference understood, camera delivery became a product setting rather than a hidden engineering assumption. OpenCV remained the compatible default. Direct V4L2, one or two buffers, supported frame rates, automatic or fixed exposure, and gain became capability-detected options. A guided camera test lets different cameras recommend supported settings instead of assuming the Razer Kiyo Pro's behaviour applies universally.

Routine Dashboard detail was throttled and moved to a latest-only worker; important state changes still publish immediately. Turning statistics off now removes that optional work rather than merely hiding already-computed detail.

Camera recovery also became evidence-driven. Reappearance in `lsusb` is not enough: recovery succeeds only after the application can read a test frame. When a hub advertises per-port power switching, `uhubctl` can perform a genuine bounded port cycle. Otherwise the helper uses safe software recovery and asks for physical reconnection when required. Saved camera controls are reapplied after restart or reconnection.

Milestone 9 - Establish the production pipeline boundary - 10 September 2026

The final refinement pass asked whether camera dequeue or Linux scheduling made palm reacquisition look expensive. A sustained output-paused trace measured the stages separately. With direct V4L2 and two buffers, the camera delivered about 30 dequeues per second:

Stage	p50 / p95
Camera dequeue cadence	33.29 / 35.84 ms
MJPEG decode	5.45 / 9.87 ms
Decode to inference pickup	18.18 / 33.27 ms
Post-graph recognition and send preparation	2.93 / 3.13 ms

Ordinary landmark continuation measured 36.86 ms median. Sixty palm-detector frames measured 104.94 / 112.89 ms p50/p95 and skipped two application-visible camera frames at median. Inference accumulated only 132 ms of run-queue wait across the entire 60-second trace. Scheduling was not the cause; the delay began inside MediaPipe's detector recovery.

Lowering tracking confidence to 0.10 and 0.20 did not prevent the same eight fast-sweep misses and worsened reacquisition p95 from about 102 ms to 126-128 ms. The accepted production boundary therefore remained:

- MediaPipe 0.10.35 with four XNNPACK CPU threads.
- 0.35 tracking confidence and 0.45 palm-detection confidence.
- Fused full-colour 640×480 preparation.
- The 2.25 direction-aware search region.
- Latest-coordinate native X/Y with reach clamping and brief-loss recovery.

Further improvement requires a different detector architecture. More threshold tuning could weaken tracking without reducing the measured detector cost, so it was stopped rather than promoted for its own sake.

Milestone 10 - Turn the prototype into a releasable product - 10-11 September 2026

The final days treated usability and reliability as parts of responsiveness. Joystick movement adopted one per-player centre box with eight-direction output outside it and immediate positional release inside it. Menu Guard, holds, turbo, special Programs, and native X/Y kept their existing priorities. The Dashboard showed useful program details when statistics were hidden.

Glove Academy became a family-friendly learning and personalization system with Pixel Pal guidance, deterministic lesson controls, dedicated gesture artwork, camera-quality advice, and controller output paused during practice. The dot test, camera wizard, hand-setup backups, and engineering package separated ordinary calibration tools from deeper tracing and replay utilities.

Installation gained repeatable two-device packages, persistent settings, precompiled Matrix firmware, early Matrix startup, mDNS resolution, automatic receiver recovery, camera recovery, backup rotation, and checks that distinguish service reachability from proven gameplay. Documentation was divided into new-user guides, current architecture, configuration reference, technical evidence, and this historical journey.

The project was then renamed VirtualGlove to distinguish the new camera-based system while retaining historically accurate references to the original Power Glove and Super Glove Ball.

The resulting engineering method

In one intensive week, a camera tracker became a two-device controller with shared recognition, joystick and native emulation, family-friendly learning, authenticated transport, resilient startup and recovery, installers, Help, and repeatable validation. The pace came from treating uncertainty as a sequence of small decisions rather than one large invention.

The loop that worked was:

1. State one observable problem in gameplay.
2. Form one testable hypothesis about a specific pipeline stage.
3. Instrument that stage without placing routine diagnostics in its critical path.
4. Replay identical input through the current and candidate lanes.
5. Reject candidates that improve one number by sacrificing continuity, correctness, stability, or understandable behaviour.
6. Deploy the safer survivor and validate it in the actual game.
7. Keep the known-working fallback until the new path proves itself.
8. Remove experimental user choices once evidence converges.

Several principles followed from that process:

- A millisecond matters in embedded work, but only after the largest costs are identified.
- The newest trustworthy measurement is more valuable than extra synthesized updates that can diverge from the hand.
- Reacquisition, stopping, direction, and continuity matter as much as average inference speed.
- A lower mathematical error does not guarantee a controller feels better.
- Technical notes are hypotheses until ROM behaviour, traces, or controlled experiments confirm them.
- Configuration switches are useful scientific controls but poor permanent substitutes for a proven default.
- Human play tests judge attachment and confidence; instrumentation explains why.

That method-not any single filter, model, emulator, or benchmark-is the most reusable result of the engineering journey.